

Educação
Profissional
Paulista

Técnico em
Ciência de
Dados

Manipulação de arquivos

Leitura de arquivos em Python

Aula 2

Código da aula: [DADOS]ANO1C2B2S16A2



Objetivo da Aula

- Aplicar os conceitos de tratamento de exceções.



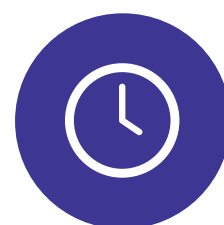
Competências da Unidade (Técnicas e Socioemocionais)

- Ser proficiente em linguagens de programação para manipular e analisar grandes conjuntos de dados;
- Usar técnicas para explorar e analisar dados, aplicar modelos estatísticos, identificar padrões, realizar inferências e tomar decisões baseadas em evidências;
- Cooperar com profissionais da área, incluindo cientistas e engenheiros de dados, e atuar em equipes diversificadas, colaborando com parceiros, supervisores e clientes.



Recursos Didáticos

- Recurso audiovisual para a exibição de vídeos e imagens;
- Acesso ao laboratório de informática e/ou à internet;
- Software Anaconda/Jupyter Notebook instalado ou similar.



Duração da Aula

50 minutos.

Exercícios de fixação: tratamento de erros

Exercício 1: corrija o código para lidar com a exceção e exibir uma mensagem amigável.

```
# Exercício 1: Corrija o código para lidar com a exceção e exibir uma mensagem amigável.  
def dividir_numeros(a, b):  
    resultado = a / b  
    print(f"Resultado da divisão: {resultado}")  
  
# Chamada da função  
dividir_numeros(10, 0)
```

Exercício 2: adicione um bloco 'try' e 'except' para tratar possíveis erros durante a conversão.

```
# Exercício 2: Adicione um bloco 'try' e 'except' para tratar possíveis erros durante a conversão.  
entrada = input("Digite um número: ")  
numero = int(entrada)  
print(f"O dobro do número é: {numero * 2}")
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Exercícios de fixação: tratamento de erros

Exercício 3: lide com a exceção `FileNotFoundError` e exiba uma mensagem personalizada.

```
# Exercício 3: Lide com a exceção FileNotFoundError e exiba uma mensagem personalizada.  
with open("arquivo_inexistente.txt", "r") as arquivo:  
    conteudo = arquivo.read()  
    print(conteudo)
```

Exercício 4: utilize um bloco `'try', 'except' e 'finally'` para garantir que o arquivo seja fechado corretamente.

```
# Exercício 4: Utilize um bloco 'try', 'except' e 'finally' para garantir que o arquivo seja fechado corretamente.  
arquivo = open("dados.txt", "w")  
arquivo.write("Conteúdo de exemplo.")  
# Realize a leitura do arquivo aqui.
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Exercícios de fixação: tratamento de erros

Exercício 5: adicione um bloco 'try', 'except' para evitar a divisão por zero e exiba uma mensagem apropriada.

```
# Exercício 5: Adicione um bloco 'try', 'except' para evitar a divisão por zero e exiba uma mensagem apropriada.  
def calcular_media(valores):  
    total = sum(valores)  
    media = total / len(valores)  
    print(f"Média: {media}")  
  
# Chamada da função  
calcular_media([])
```

Exercício 6: identifique e trate a exceção para que o programa não seja encerrado abruptamente.

```
# Exercício 6: Identifique e trate a exceção para que o programa não seja encerrado abruptamente.  
lista = [1, 2, 3]  
print(lista[5])
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Exercícios de fixação: tratamento de erros

Exercício 7: adicione um bloco `'try', 'except'` para tratar uma possível exceção durante a conversão.

```
# Exercício 7: Adicione um bloco 'try', 'except' para tratar uma possível exceção durante a conversão.  
idade = input("Digite sua idade: ")  
idade_numerica = int(idade)  
print(f"No próximo ano, você terá {idade_numerica + 1} anos.")
```

Exercício 8: utilize um bloco `'try', 'except'` para lidar com a exceção durante a execução da divisão.

```
# Exercício 8: Utilize um bloco 'try', 'except' para lidar com a exceção durante a execução da divisão.  
numero1 = 10  
numero2 = "5"  
resultado = numero1 / numero2  
print(f"Resultado da divisão: {resultado}")
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Exercícios de fixação: tratamento de erros

Exercício 9: corrija o código para lidar com a exceção durante a leitura do arquivo.

```
# Exercício 9: Corrija o código para lidar com a exceção durante a leitura do arquivo.  
with open("arquivo_corrompido.txt", "r") as arquivo:  
    conteudo = arquivo.read()  
    print(conteudo)
```

Exercício 10: adicione blocos **'try'**, **'except'** para tratar possíveis erros durante a execução do código.

```
# Exercício 10: Adicione blocos 'try', 'except' para tratar possíveis erros durante a execução do código.  
valor1 = int(input("Digite um número: "))  
valor2 = int(input("Digite outro número: "))  
resultado = valor1 / valor2  
print(f"Resultado da divisão: {resultado}")
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Vamos
fazer uma
atividade

Na documentação de Python, o capítulo 8 é focado nos erros e nas exceções. Vamos realizar uma síntese sobre a ideia que cada tópico transmite?

 20 minutos

 Em grupo

Resumo: tratamento de erros e exceções

Leia o seguinte conteúdo sobre erros e exceções:

PYTHON. 3.12.2 *Documentation*. O tutorial de Python. 8.

1 Erros e exceções. Disponível em:
<https://docs.python.org/pt-br/3/tutorial/errors.html>.
Acesso em: 5 abr. 2024.

2 A partir da leitura, faça uma síntese dos **dez tópicos** apresentados na documentação do Python.

Estruture seu texto de **maneira clara e lógica**.

3 Comece com uma introdução ao tema, seguida de suas reflexões, e conclua com uma ideia final.

Após finalização do resumo, encaminhe o registro no AVA (Ambiente Virtual de Aprendizagem).



O que nós
**aprendemos
hoje?**

© Getty Images

Hoje desenvolvemos:

- 1** A compreensão sobre a importância do tratamento de erros a partir da necessidade de identificar e lidar com exceções em programas de análise de dados.
- 2** A habilidade em implementar as estruturas **try, except, else** e **finally**, aprimorando sua capacidade de escrever códigos que podem se recuperar de falhas inesperadas.
- 3** O domínio de técnicas de prevenção e diagnóstico de erros antes que ocorram e em métodos que facilitam a rápida identificação de problemas.



Saiba mais

Imagine você se deparar com o desafio de lidar com erros e exceções em Python. Leia o artigo indicado abaixo e veja como agir:

ORESTES, Y. Python: Lidando com erros e exceções. *Alura*, 20 nov. 2018. Disponível em: <https://www.alura.com.br/artigos/tratamento-de-excecoes-no-python>. Acesso em: 5 abr. 2024.

Referências da aula

MENEZES, N. N. C. *Introdução à programação com Python: algoritmos e lógica de programação para iniciantes*. São Paulo: Novatec, 2019.

ORESTES, Y. Python: Lidando com erros e exceções. *Alura*, 20 nov. 2018. Disponível em: <https://www.alura.com.br/artigos/tratamento-de-excecoes-no-python>. Acesso em: 5 abr. 2024.

PYTHON. 3.12.2 *Documentation*. O tutorial de Python. 8. Erros e exceções. Disponível em: <https://docs.python.org/pt-br/3/tutorial/errors.html>. Acesso em: 5 abr. 2024.

Identidade visual: Imagens © Getty Images

Educação
Profissional
Paulista

Técnico em
Ciência de
Dados